Ders 6 Genişletilmiş Notu: Register Transferi ve Ortak Veri Yolu, Komut Kümesi Mimarisi, Adresleme Modları ve Koşullu Dallanma

Ders 6 Genişletilmiş Notu: Register Transferi ve Ortak Veri Yolu, Komut Kümesi Mimarisi, Adresleme Modları ve Koşullu Dallanma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Hedefi: Bir CPU'nun İki Bloğu
- 2. Ayrılmış Çoklayıcı (Dedicated Multiplexer)
- 3. Tek Veri Yolu (Single Bus)
- 4. Üç Durumlu Tampon (Three-State Buffer)
- 5. Üç Durumlu Veri Yolu (Three-State Bus)
- 6. Komut Kümesi Mimarisine Giriş
- 7. Komut Biçimi ve Komut Çevrimi
- 8. Tasarlanacak Basit Mimarinin Register'ları
- 9. Açık ve Örtük Adres
- 10. Adres Sayısına Göre Komutlar
- 11. Yığın İşlemleri
- 12. Adresleme Modları
- 13. RISC ve CISC Karşılaştırması
- 14. Veri Transfer Komutları ve Giriş-Çıkış Adresleme
- 15. Veri İşleme Komutları
- 16. Program Kontrol Komutları
- 17. Koşullu Dallanma Komutları ve Durum Bitleri
- 18. Çözümlü Örnek 1: İşaretsiz Karşılaştırma
- 19. Çözümlü Örnek 2: Aritmetik Kaydırma ve İşaretli Karşılaştırma
- 20. Ders Sonu Sorusu: Onluk Tabanda Tümleyen

1. Dersin Hedefi: Bir CPU'nun İki Bloğu

Bu dersin iki bölümü var. İlkinde register'lar üzerindeki transfer işlemlerini ve ortak veri yolu oluşturmayı, ikincisinde komut yapısını ve adresleme yöntemlerini göreceğiz. İkinci bölüm, assembly dersini alanlara kısmen tanıdık gelecek.

Bütün bu altyapının amacı bir işlemci tasarlamaktır. Merkezi işlem biriminde (CPU) `MOVE AX, BX` gibi bir komut yazdığınızda iki register arasında veri transferi yapılır. Bu transfer, işlemcinin içindeki iç veri yolu (internal bus) üzerinden gerçekleşir. İç veri yolu, bütün register'ların bağlı olduğu ortak bir yapıdır. Hoca bu derste bu yapının nasıl tasarlandığını, en ilkel biçiminden bugün kullanılan biçimine kadar, birkaç modeli avantaj ve dezavantajlarıyla karşılaştırarak anlattı. Slaytlar Mano'nun kitabının şekillerinden oluşuyor ve ilgili konu kitabın yedinci bölümünde.

Tasarlamayı hedeflediğimiz CPU iki bloktan oluşur:

- **Veri yolu birimi (datapath unit):** Veriyi işleyen ve register'lar üzerinde tutan birimdir. İçinde aritmetik lojik birim (ALU), kaydırma birimi (shift unit), iç register'ları tutan register dosyası (register file) ve bunlar arasında transferi sağlayan iç veri yolu bulunur.
- **Kontrol birimi (control unit):** O anda yürütülecek işleme göre kontrol işaretlerini gönderir. Toplama mı yapılacak, veri mi transfer edilecek, artırma ya da eksiltme mi yapılacak, bunu kontrol işaretleriyle söyler.

İki blok arasındaki ilişki iki yönlüdür. Kontrol birimi işaretleri gönderir. Veri yolu birimi işlemi yaptıktan sonra mevcut durumu durum bitleriyle (taşma, sıfır, negatif, elde: overflow, zero, negative, carry) ve kontrol için kullanılan başka bitlerle kontrol birimine bildirir. Kontrol birimi bir sonraki işlem adımına buna göre karar verir.

Bu derste veri yolu biriminin içindeki register'ların iç veri yolu yapısına odaklanıyoruz. ALU ve kaydırma birimi tasarımı, sonra da kontrol birimi daha sonraki konulardır.

Register tasarımlarından bir hatırlatma da yapıldı: Bir register'a yeni değer yüklemek için saat (clock) girişinin önüne bir devre yerleştirilmemelidir. Çünkü amacımız birden fazla birimi senkron çalıştırmaktır. Yükleme, register'ın içindeki D tipi flip-flop'un D girişinin önüne yerleştirilen bir devreyle yapılır. Register ve sayaç tasarımıyla ilgili slaytlar önceki haftalarda işlendiği için atlandı.

2. Ayrılmış Çoklayıcı (Dedicated Multiplexer)

Anlatım kolay olsun diye sistemde toplam üç register olduğunu kabul edelim: R_0, R_1 ve R_2. Amacımız bunlar arasında veri transferi yapmaktır.

Akla gelen en temel çözüm şudur: R_0'a R_1'den ya da R_2'den veri gelebilir. R_1'e R_0'dan ya da R_2'den veri gelebilir. Yani her register'ın kendisi dışında iki kaynağı vardır. O hâlde her register'ın önüne, bu kaynaklar arasında seçim yapan bir 2'den 1'e çoklayıcı (multiplexer) yerleştirilir.

Örneğin R_0'ın önündeki çoklayıcıda R_1'in çıkışı 1 numaralı girişe, R_2'nin çıkışı 0 numaralı girişe bağlıdır. Çoklayıcının bir seçme ucu vardır. Seçme ucu 0 ise 0 numaralı giriş, 1 ise 1 numaralı giriş çıkışa aktarılır.

Üç register'ın önünde üç çoklayıcı olduğu için kaynak seçen üç seçme ucu vardır. Bunlara ek olarak her register'ın bir yükleme ucu (load enable) bulunur. Yükleme uçları hangi register'lara yükleme yapılacağını belirler. Yalnız bir register'a ya da aynı anda ikisine transfer yapılabilir.

Yöntemin adı buradan gelir: *Ayrılmış*, yani her register'ın önünde yalnız ona ait bir çoklayıcı bulunur.

- **Avantajı:** Esnektir. Aynı anda farklı kaynaklardan farklı hedeflere transfer yapılabilir.
- **Dezavantajı:** Pahalıdır. Üç register için üç çoklayıcı çok maliyetli görünmeyebilir. Ama iç register sayısı 15-20 olsaydı her register'ın önüne bir çoklayıcı koymak gerekirdi ve bu çoklayıcılar 16'dan 1'e, 32'den 1'e gibi çok daha büyük olurdu. Bu yöntem bir çözüm sunar, ama çok maliyetli olduğu için tercih edilmez.

3. Tek Veri Yolu (Single Bus)

Alternatif çözümde yine üç register vardır, ama önlerinde tek bir ortak çoklayıcı bulunur. R_0 çoklayıcının 0, R_1 1, R_2 2 numaralı girişine bağlıdır.

Slayt bunu "3'ten 1'e çoklayıcı" diye adlandırır. Hoca bunun standart bir ölçü olmadığını, aslında bir 4'ten 1'e çoklayıcı olduğunu belirtti: İki seçme ucu ve dört girişi vardır, ama 3 numaralı giriş kullanılmaz.

Burada dikkat edilecek nokta, register'ların n bitlik olmasıdır. Yani çoklayıcı tek tek bitler arasında değil, n bitlik sayılar arasında seçim yapar. (Bu, önceki yapı için de geçerliydi.)

Çoklayıcının çıkışı bütün register'ların n bitlik veri girişine bağlanır. Böylece kaynak olarak seçilen register'ın içeriği tek veri yolu üzerinden bütün register'ların girişine gider. Ama yükleme yalnız yükleme ucu 1 olan register'lara yapılır. Kısacası **seçme uçları kaynağı, yükleme uçları hedefi belirler**.

3.1 Transfer Örnekleri

Örnek 1: $R_0 \leftarrow R_2$**.** Hedef R_0, kaynak R_2'dir. R_2'yi seçmek için $S_1S_0 = 10$ olmalıdır. Yalnız R_0'a yükleme yapılacağı için yalnız L_0 1, öbürleri 0 olmalıdır: $L_2L_1L_0 = 001$.

Örnek 2: $R_0 \leftarrow R_1$ **ve** $R_2 \leftarrow R_1$ **aynı anda.** İki transferin kaynağı aynı register olduğu için bu işlem tek veri yolunda yapılabilir. R_1'i seçmek için yüksek anlamlı seçme ucu 0, düşük anlamlı seçme ucu 1 olur: $S_1S_0 = 01$. Böylece ortak veri yolunda R_1'in değeri oluşur. Hedefler R_2 ve R_0 olduğu için $L_2L_1L_0 = 101$ yazılır.

Örnek 3: $R_0 \leftarrow R_1$ **ve** $R_1 \leftarrow R_0$ **aynı anda.** Hoca bu örneği soru olarak sordu. Bu işlem tek veri yolunda **yapılamaz**, çünkü iki farklı kaynak vardır ve veri yolu bir anda yalnız bir register'ın n bitlik içeriğini taşıyabilir. Oysa ayrılmış çoklayıcı yapısında bu mümkündü: Her register kendi çoklayıcısıyla kendi kaynağını seçer.

İstenen transfer	S_1S_0	$L_2L_1L_0$	Tek veri yolunda
$R_0 \leftarrow R_2$	10	001	Yapılır
$R_0 \leftarrow R_1$, $R_2 \leftarrow R_1$	01	101	Yapılır
$R_0 \leftarrow R_1$, $R_1 \leftarrow R_0$	-	-	Aynı anda yapılamaz

Soru: Bir sonraki saat periyodunda ikinci işlemi yapabiliriz, değil mi? **Cevap:** Tabii ki yapabiliriz, burada aynı anda yapmaktan söz ediyoruz. Tek veri yolu yapısı işlemlerin art arda yapılmasını gerektirir. Ayrılmış çoklayıcıya göre dezavantajı budur.

> **Ders sonrası not:** Hocanın cevabı iki transferin ayrı periyotlarda yapılabileceği içindir. Ama sonuç, aynı anda yapılan takasla aynı olmaz. Önce $R_0 \leftarrow R_1$ yapılırsa R_0'ın eski değeri kaybolur, sonraki $R_1 \leftarrow R_0$ adımı R_1'e yine kendi eski değerini yazar. Python benzetiminde iki adımın sonunda her iki register'da da R_1'in eski değeri kaldı. Takas için geçici bir alan gerekir. Hoca bu üç adımlı yöntemi Bölüm 14'te exchange komutunu anlatırken verdi.

Derste bir öğrencinin isteğiyle iki yapı kısaca tekrar edildi: Soldaki yapıda her register'ın önünde bir çoklayıcı vardır ve daha esnektir. Sağdaki yapıda sistemde tek çoklayıcı vardır, bütün register çıkışları onun girişleridir. Seçme uçlarıyla kaynak seçilir, yükleme uçlarıyla hedefler etkinleştirilir.

4. Üç Durumlu Tampon (Three-State Buffer)

Şimdi daha önce gördüğümüz bir devre elemanıyla, üç durumlu tamponla, üçüncü bir veri yolu yapısı kuracağız. Önce elemanı hatırlayalım.

Üç durumlu tamponun bir girişi, bir çıkışı ve bunları kontrol eden bir etkinleştirme (enable) ucu vardır:

E	Giriş	Çıkış
0	X	High-Z
1	0	0
1	1	1

$E = 0$ ise giriş ne olursa olsun çıkış High-Z durumundadır. $E = 1$ ise eleman tampon gibi davranır, girişte ne varsa çıkışa aktarır. Önemli olan üçüncü durum High-Z'dir. "Üç durumlu" adı da buradan gelir. High-Z yüksek direnç demektir: Çıkışta çok yüksek direnç varsa devrenin o kolu, dışarıdan bakıldığında sanki kopartılmış gibi davranır.

4.1 Çıkışları Birbirine Bağlamak

Normal lojik kapıların çıkışları birbirine doğrudan bağlanamaz. Örneğin bir AND kapısı ile bir OR kapısının çıkışını birbirine kısa devre edemezsiniz. Böyle bir birleştirme gerekiyorsa iki çıkış yeni bir kapıda birleştirilir.

Nedeni şudur: AND kapısı lojik 1, yani +5 V üretirken OR kapısının çıkışı lojik 0 ise +5 V'tan 0'a doğru bir akım akar. Hocanın deyişiyle OR kapısının üstünden dumanlar çıkar, kapı yanar.

Üç durumlu tamponlarda ise bu bağlantı, bir şartla, yapılabilir. İki tamponun çıkışını birbirine bağlayalım. Üstteki tamponun girişi I_0, etkinleştirme ucu E_0; alttakinin girişi I_1, etkinleştirme ucu E_1 olsun. Durumlar şöyledir:

E_1	E_0	I_1	I_0	Çıkış	Yorum
0	0	X	X	High-Z	İki kol da kopuk gibi
0	1	X	0	0	Üstteki aktarır
0	1	X	1	1	Üstteki aktarır
1	0	0	X	0	Alttaki aktarır
1	0	1	X	1	Alttaki aktarır
1	1	0	0	0	Girişler aynı, sorun yok
1	1	1	1	1	Girişler aynı, sorun yok
1	1	0	1	Çakışma	Yüksek akım, devre zarar görür
1	1	1	0	Çakışma	Yüksek akım, devre zarar görür

İkisi birden etkinse ve girişleri aynıysa çıkış sorunsuz çalışır. Ama girişlerden biri 1, öbürü 0 ise az önceki problem yeniden doğar: Yüksek akım akar, tamponun üstünden dumanlar çıkar ve devre yanar.

> **Hocanın vurgusu:** Bu bağlantı yalnız aynı anda en çok bir tampon etkin olursa kullanılabilir. Aynı anda ikisinin birden etkin olmamasını garanti etmeniz gerekir.

5. Üç Durumlu Veri Yolu (Three-State Bus)

Bu elemanı register'lar için ortak veri yolu kurarken şöyle kullanırız: Her register'ın n bitlik çıkışına üç durumlu tampon yerleştirilir. Tamponun çıkışı veri yoluna, veri yolu da bütün register'ların girişine bağlanır. Slayttaki adı "çift yönlü giriş-çıkış hatlarına sahip register"dır (register with bidirectional input-output lines).

> **Hocanın vurgusu:** "Nasıl çift yönlü oluyor biliyor musunuz? Lütfen burayı çok iyi dinleyin."

Register'ın etkinleştirme ucu 1 ise tampon register'ın içindeki sayıyı dışarı verir. Bu hat üzerinde çıkış yönünde bir veri akışıdır. Ama birden fazla register'ı aynı hatta bağladığımızda bir register'ın çıkışı öbür register'ların girişi olur. Yani aynı hat, hem giriş hem çıkış yönünde çalışır. Çift yönlü olmanın anlamı budur.

Çalışma örneği: Veri yoluna veriyi gönderecek register R_0 olsun.

1. $E_0 = 1$ yapılır, öbür etkinleştirme uçları 0 kalır.
2. R_0'ın çıkışındaki tampon yolu açar ve R_0'ın içeriği veri yoluna bırakılır.
3. Bu değer veri yolu üzerinden bütün register'ların girişine gider.
4. Yükleme ucu 1 yapılmış olan register'lar bu değeri yükler.

Kısacası çıkış verip vermeyeceğine E ucu, yükleyip yüklemeyeceğine L ucu karar verir. Çift yönlü yapıyı gösterirken bir üçgen sembolü kullanılır. Bunu bellek tasarımında da görmüştük: Üç durumlu çıkışı gösterir ve bu çıkışların birbirine bağlanabileceğini anlatır.

5.1 Etkinleştirme Uçlarını Bir Decoder Kontrol Eder

Aynı anda veri yoluna iki register veri göndermek isterse ve biri lojik 1, öbürü lojik 0 aktarırsa devre zarar görür. Hocanın deyişiyle yapı çöker. Bunun önüne geçmek için etkinleştirme uçlarını bir decoder kontrol eder: Decoder yalnız bir çıkışını 1 yaptığı için aynı anda yalnız bir register veri yoluna veri gönderir. Slayttaki şekilde bu decoder çizilmemiştir.

> **Hocanın vurgusu:** Üç durumlu veri yolu bu bölümün önemli konusudur. “Bunu anlamanızı özellikle istiyorum.” Bundan sonraki amacımız CPU tasarımıdır ve bu yapı işlemci tasarımında özellikle kullanılan yapılardan biridir.

5.2 Soru-Cevaplar

Soru: Bir register’ın etkinleştirme ucu 1 olduğunda, değeri tutmak için kendi yükleme ucunun da açık olması gerekir mi? **Cevap:** Hayır, gerekmez. Register değeri saklamaya devam eder. D tipi flip-flop’lu register tasarımını hatırlayın: Q çıkışları register’ın çıkışıdır. Biz yalnız bu çıkışların peşine bir üç durumlu tampon koyduk. Tampon, çıkışın dışarı aktarılıp aktarılmayacağını seçer. Tamponun çıkışı hem kendi register’ına hem öbür register’lara giriş olarak gider, ama yükleme yalnız o register’ın yükleme ucu 1 ise olur.

Soru: L_0 ve E_0 aynı anda 1 olursa ne olur? **Cevap:** R_0’ı veri yoluna veriyorsunuz ve aynı register’a yeniden yükleme yapıyorsunuz. Bu yapılabilir. Register’lar saatle senkron çalıştığı için önceki durumdaki Q değeri, saatin bir sonraki periyodunda kendi üzerine yeniden yüklenmiş olur. Yüklenen değer zaten aynı değerdir. Bu sırada öbür register’lara da yükleme yapılabilir.

Soru: Şekildeki (c) yapısının giriş ve çıkışları (b) yapısındaki gibi değil mi? Aralarındaki en büyük fark ne? **Cevap:** Evet, en büyük fark aradan çoklayıcının çıkmasıdır. Artık kaynağı, etkinleştirme uçlarına verilen değerlere göre seçiyoruz. Sanki çoklayıcının seçme uçlarının yerine etkinleştirme uçları gelmiştir.

6. Komut Kümesi Mimarisine Giriş

Dersin ikinci bölümü komut kümesi mimarisi (Instruction Set Architecture, ISA), yani komut seti ve adresleme yöntemleridir. Assembly dersinde büyük ölçüde 8086 mimarisinin komut setini öğrendiniz. Bu dersin amacı ise tek bir işlemciyi değil, genel yapıyı anlatmaktır. Farklı işlemcilerle karşılaşabilirsiniz, bu yüzden geliştirilmiş yapıların birbirlerine göre avantajlarını görmek önemlidir.

> **Hocanın vurgusu:** Bu ders 8086 assembly dersi değildir, bilgisayar mimarisi yapılarını öğreten bir derstir. Genel kavramları bilirseniz bugün 8086 ile çalışırken yarın başka bir işlemciye geçtiğinizde zorlanmazsınız. Bu yüzden tek bir pencereden değil, daha geniş bir pencereden bakacağız.

Bölümün başlıkları şunlardır: işlem operandlarının nasıl adreslendiği, ardından temel komutlar olarak veri transfer, veri işleme ve program kontrol komutları.

Yüksek seviyeli bir dille yazdığımız kodu derleyici (compiler) assembly diline çevirir. Assembler bunu makine koduna çevirir. İşlemci, sıfır ve birlerden oluşan bu makine kodunu anlayıp gerekli kontrol işaretlerini oluşturarak ilgili işlemi gerçekleştirir.

Bir mimari tasarlarken en başta bir komut seti oluşturulur. Sonra bu komut setini gerçekleştirebilecek donanım yapısı kurulur.

6.1 CISC ve RISC

Komut kümesi mimarisi başlığı altında temel olarak iki yapıdan söz edilir:

- **CISC (Complex Instruction Set Computer):** Öğrendiğiniz 8086 bu mimaridedir. Çarpma, bölme gibi karmaşık komutlar da vardır. En önemli özelliği, kontrol devresinin yazılımsal, yani mikroprogramlı olmasıdır.
- **RISC (Reduced Instruction Set Computer):** Daha az sayıda ve daha basit komut vardır. Kontrol devresi kablolu (hardwired) olarak kurulur ve daha hızlı çalışır. Çarpma, bölme gibi karmaşık işlemler basit komutlar art arda getirilerek bir programla yapılır, bu da işlem sayısını artırır. Hoca RISC’in içinde çarpma ve bölme komutlarının olmadığını, daha az sayıda iç register bulunduğunu da söyledi.

  Ders sonrası not: Hoca RISC’te daha az sayıda iç register olduğunu ve çarpma-bölme komutu bulunmadığını söyledi. Altyazıda bunu sınırlayan bir açıklama yok. Bu iki özellik her RISC işlemci için geçerli değildir: RISC işlemcilerde register sayısı büyük olasılıkla CISC’tekinden çoktur, birçok RISC işlemcide de büyük olasılıkla çarpma komutu vardır. Kitaptaki tanımla karşılaştırın.

Soru: CISC komutları daha uzun yazılıyor ve tek bir komutun içinde başka başka işlemler mi oluyor? **Cevap:** Evet, doğru. Komut kodu da daha uzundur ve değişik özellikleri vardır.

7. Komut Biçimi ve Komut Çevrimi

7.1 Komut Biçimi

Örneğin 2 ya da 3 bayt uzunluğunda bir komut düşünelim. Bir komutun içinde şu bölümler bulunur:

- **İşlem kodu (opcode, operation code):** Yürütülecek komutun ne olduğunu söyler. Transfer mi, toplama mı, artırma ya da eksiltme mi?
- **Adres alanı (address field):** İşlemde kullanılacak operandın adresini gösterir. Birden fazla olabilir. Bazı yapılarda hedef, kaynak 1 ve kaynak 2 olmak üzere üç adres alanı vardır.
- **Mod alanı (mode field):** Adresleme modu bilgisini tutar.
- **Diğer alan (other field):** Gerekirse kullanılır.

7.2 Komut Çevrimi

Bir komutun tamamlanması için art arda yapılması gereken adımlar vardır. Slayttaki adı işlem çevrimidir (operation cycle):

1. **Komut getirme (Instruction Fetch, IF):** Yürütülecek komut bellektedir. Komut register'ına (Instruction Register, IR) getirilir.
2. **Komut çözme (Instruction Decode, ID):** Opcode alanına bakılır. Komutun toplama mı, çıkarma mı yapacağı ve hangi adresleme modunu kullandığı çözülür.
3. **Operand getirme (Operand Fetch, OF):** Adresleme moduna göre operandın nereden alınacağına karar verilmiştir. İşleme katılacak sayı bellekteyse oradan getirilir.
4. **Yürütme (Execute, EX):** Operand hazırdır, yapılacak işlem bellidir. Aritmetik ya da transfer işlemi yürütülür.
5. **Sonucu yazma (Write Back, WB):** Aritmetik işlemin sonucu bir register'a ya da belleğe kaydedilir. Slaytta adı "result store"dur.
6. **Sonraki komut:** Bir sonraki komutu gösteren işaretçi ayarlanır, çevrim başa döner ve aynı adımlar sonraki komut için tekrarlanır.

   Hocanın vurgusu: Bu adımları bilmekte fayda var. İşlemi paralelleştirmek için bu yapıyı ileride ardışık düzen (pipeline) işleme bölümünde ayrıntılı olarak kullanacağız.

8. Tasarlanacak Basit Mimarinin Register'ları

Bu ders kapsamında basit bir mimari tasarlamaya çalışıyoruz. Bu mimarinin register'ları şunlardır:

- **Register seti:** 16 register vardır. R_0-R_7 programcının erişebildiği register'lardır (programmer accessible registers). R_8-R_{15} yalnız içeride, mikro işlemler ve mikro programlar için kullanılır.
- **Komut register'ı (IR):** Komut getirme adımında bellekten alınan komutun yazıldığı register'dır. Çözme işlemine buradaki komut girer.
- **Program sayacı (Program Counter, PC):** Yürütülecek komutun bellekte hangi adreste tutulduğunu gösterir. Programın bir satırındaki komut işlenince PC artırılır ve sonraki satırdaki komut yürütülür. "Sayaç" denmesinin nedeni bu sayma özelliğidir.
- **Pipeline register'ları:** İleride ele alınacak.
- **Durum register'ı (status register):** Carry, overflow, negative ve zero gibi bayrakları durum biti olarak tutar.
- **Yığın işaretçisi (stack pointer):** Yığın işlemlerinde kullanılır.

  Ders sonrası not: Hoca PC'nin "bir artırıldığını" söyledi. Bu, tek gözlük komutlar için doğrudur. Komut birden çok bellek gözü kaplıyorsa PC komutun uzunluğu kadar ilerler. Bölüm 12'de hocanın kendi örneğinde 250-251 adreslerindeki komut getirildikten sonra PC 252 olur.

9. Açık ve Örtük Adres

Operand adreslemede iki kavram vardır:

- **Açık adres (explicit address):** Adres bilgisi komutun bir parçası olarak adres alanında verilmiştir.
- **Örtük adres (implied address):** Operandın yeri opcode tarafından belirtilir, yani opcode'un içinde gizlidir. Hoca bunun yalnız akümülatör register'ının olduğu durumlarda kullanıldığını söyledi. Örneğin "akümülatöre yükle" (load to accumulator) komutu, işlemin akümülatör üzerinde yapılacağını zaten opcode'un içinde söyler.

  Ders sonrası not: "Yalnız akümülatör" ifadesi bu örnek için geçerlidir. Bölüm 10'daki sıfır adresli komutlarda da operandların yeri komutta yazılmaz, yığından anlaşılır. Örtük adres büyük olasılıkla akümülatörle sınırlı değildir.

10. Adres Sayısına Göre Komutlar

Farklı mimarilerde komutların içinde farklı sayıda adres alanı bulunur. Slayt bunları aynı problem üzerinde karşılaştırır: Bellekte A, B, C, D sayıları vardır. $A + B$ ve $C + D$ hesaplanacak, sonuçlar çarpılacak ve çarpım bellekte X adresine yazılacaktır:

$$X = (A + B)(C + D)$$

10.1 Üç Adresli Komutlar

Komutta hedef, kaynak 1 ve kaynak 2 adresleri bulunur:

```
ADD T1, A, B     ; M[T1] <- M[A] + M[B]
ADD T2, C, D     ; M[T2] <- M[C] + M[D]
MUL X, T1, T2    ; M[X]  <- M[T1] * M[T2]
```

A ve B gözlerindeki değerler toplanır, sonuç geçici (temporary) T_1 adresine yazılır. C ve D için de aynısı yapılır, sonuç T_2'ye yazılır. Sonra iki geçici alan çarpılıp X adresine yazılır. İşlem 3 satırda biter. Slayt aynı işlemin register'larda tutularak da yapılabileceğini gösterir.

- **Avantajı:** Program kısadır.
- **Dezavantajı:** Üç adresi belirtmek için ikili kodlanmış komut daha çok bit ister, yani komutlar uzundur.

10.2 İki Adresli Komutlar

İki adres alanı vardır ve bunlardan biri hem kaynak hem hedeftir:

```
MOVE T1, A       ; M[T1] <- M[A]
ADD  T1, B       ; M[T1] <- M[T1] + M[B]
MOVE X, C        ; M[X]  <- M[C]
ADD  X, D        ; M[X]  <- M[X] + M[D]
MUL  X, T1       ; M[X]  <- M[X] * M[T1]
```

Soru (hocanın sorusu): İkinci satırda görüldüğü gibi iki bellek gözü arasında zaten toplama yapılabiliyor. Öyleyse birinci satırda A gözündeki değer neden T_1'e taşındı? **Cevap (bir öğrenci):** $M[A]$'nın değerini bozmamak için. ADD sonucu ilk operandın üstüne yazar. Hoca cevabı doğruladı.

10.3 Bir Adresli Komutlar

Burada örtük adres devreye girer. Akümülatör adı verilen özel bir register vardır ve işlemler hep onun üzerinden döner. Bir operand ve hedef akümülatördür. Bu, opcode alanında belirtilir:

```
LD  A      ; ACC <- M[A]
ADD B      ; ACC <- ACC + M[B]
ST  X      ; M[X] <- ACC
LD  C      ; ACC <- M[C]
```

```
ADD D      ; ACC <- ACC + M[D]
MUL X      ; ACC <- ACC * M[X]
ST  X      ; M[X] <- ACC
```

$A + B$ toplamı geçici olarak X gözünde depolanır. Sonra $C + D$ akümülatörde hesaplanır, X'teki değerle çarpılır ve sonuç yeniden X'e kaydedilir. Komut uzunluğu daha kısadır, ama 7 komut gerekir.

10.4 Sıfır Adresli Komutlar

Hoca bu yapıyı "Yok artık diyeceksiniz, nasıl yani?" diye tanıttı. Her şey yığına (stack) kaydedilir ve işlemler yığın üzerinde yapılır. Yığın, bellekte başlangıç noktası belli, tanımlı bir özel alandır. Ona bir işaretçiyle erişilir. Slaytta bu işaretçinin adı yığının tepesidir (top of stack, TOS). Normalde yığın işaretçisi (stack pointer) deriz.

```
PUSH A
PUSH B
ADD        ; TOS <- TOS + TOS-1
PUSH C
PUSH D
ADD
MUL
POP X
```

`PUSH A`, A gözündeki değeri yığına yerleştirir. Ardından `PUSH B` aynısını B için yapar. `ADD`, işaretçinin gösterdiği değerle bir altındaki gözdeki değeri toplar ve sonucu yine yığına yazar.

> **Ders sonrası not:** Hoca yalnız ilk adımları anlattı. Yukarıdaki tam program kısa nottan alındı. Python ile çalıştırıldığında 3, 5, 7 ve 8 komutluk dört program da $(A+B)(C+D)$ sonucunu verdi.
>
> **Ders sonrası not:** Hoca burada PUSH'tan sonra işaretçinin "bir arttığını, bir üst gözü gösterdiğini" söyledi. Bölüm 11'deki modelde ise PUSH işaretçiyi bir **azaltır**. Çizimde düşük adresler yukarıda olduğu için "bir üst göz" doğrudur, ama adres azalır.

Bu örnekler mimari çözümlerin neler olabileceğini gösteriyor. Adres sayısı azaldıkça komut kısalır, komut sayısı artar.

11. Yığın İşlemleri

Bütün yığın işlemlerinde iki temel işlem vardır: PUSH, bir register'daki sayıyı yığına depolamak (itmek) demektir. POP, yığına kaydedilmiş veriyi geri çekmek, okumak demektir.

Hoca buradaki yapının teorik bir model olduğunu, 8086 ile farklılık gösterebileceğini belirtti. Bütün mimarilerde tek tip bir yapı yoktur, ama hepsinde bir işaretçi yapısı vardır.

Modelde yığın işaretçisi (SP) 101 adresini göstermektedir. Adreslerin yönüne dikkat edin: Dolu gözler 102, 103, 104 diye aşağı doğru, boş göz 100 ise yukarı doğru çizilmiştir. Bu modelde **SP her zaman dolu olan gözü gösterir**. 101 gözü de doludur.

PUSH: Önce SP bir azaltılır, sonra o adrese R_1 yazılır. Örnekte SP 101'den 100'e iner ve 100 adresine R_1 yazılır. İşlemin sonunda SP yine dolu bir gözü, 100'ü gösterir.

$$SP \leftarrow SP - 1, \qquad M[SP] \leftarrow R_1$$

POP: Önce SP'nin gösterdiği gözden veri R_1'e alınır. SP'nin hep dolu gözü göstermesi gerektiği için sonra bir artırılır. Böylece bir önceki kayıt gösterilmiş olur.

$$R_1 \leftarrow M[SP], \qquad SP \leftarrow SP + 1$$

Slaytta yığın kullanımına örnek olarak ters Polonya gösterimi (Reverse Polish Notation) de veriliyor. Bu gösterimde verilmiş bir aritmetik işlemin yığında nasıl yapılacağı gösterilir. Hoca

bunu ayrıntılandırmadı, ayrık matematik ya da veri yapıları dersinde görülmüş olabileceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Ters Polonya gösteriminde işleç, operandlardan sonra yazılır. $(A + B)(C + D)$ ifadesi bu gösterimde $A\ B\ +\ C\ D\ +\ \times$ olur. Soldan sağa okunurken operand görülünce PUSH, işleç görülünce işlem yapıldığı düşünülürse Bölüm 10.4'teki sıfır adresli programın sırası elde edilir.

12. Adresleme Modları

Adresleme modları, bütün mimarilerde görülebilecek temel yöntemlerdir: örtük (implied), anlık (immediate), register, register dolaylı (register indirect), doğrudan (direct), dolaylı (indirect), PC'ye göreli (program counter relative) ve indisli (indexed). Hoca hepsini tek bir basit örnekle anlattı.

> **Hocanın vurgusu:** Bilimsel hazırlık programındaki öğrencilerin bu konulara ekstra çalışması gerekir. Lisans öğrencileri assembly bilir ve belli bir işlemciye özgü çözümleri çok kullanmıştır. Ama hazırlıkta bu kavramlarla ilk kez tanışılıyor olabilir.

12.1 Örneğin Kurulumu

Bellekte 250 ve 251 adreslerinde iki gözlük bir komut vardır. Komutun opcode alanı, mod alanı ve adres alanı bulunur. Opcode çözüldüğünde komutun "akümülatöre yükle" (load to accumulator) olduğu görülür. Neyin yükleneceği adresleme moduna bağlıdır. Adres alanında 500 sayısı vardır.

Hoca 250 ve 251'deki gözleri önce birer bayt diye anlattı. Sonra 500 sayısı 8 bite sığmadığı için bu gözlerin daha büyük, örneğin 16 bit olabileceğini ekledi.

Örnekte bellek ve register'ların değerleri şunlardır:

Adres ya da register	İçerik
251 (adres alanı)	500
400	700
500	800
800	300
900	200
R_1	400
İndis register'ı	400
PC (komut getirildikten sonra)	252

12.2 Modlar Tek Tek

Slaytta her mod için sembolik gösterim, register transfer ifadesi, operandın adresi ve akümülatöre yüklenen değer tablo hâlinde verilir. Hoca en açık olanın register transfer ifadesi olduğunu söyledi. Slaytta adres alanı "adres ya da sayı" (address or number) diye adlandırılır, çünkü anlamı moda göre değişir. Aşağıda $ADRS$ adres alanını, NBR sayıyı gösterir.

Doğrudan mod. Adres alanı doğrudan operandın adresidir. 500 adresine gidilir, oradaki 800 okunur ve akümülatöre yüklenir: $AC \leftarrow M[ADRS]$.

Anlık mod. Adres alanındaki değer sayının kendisidir: $AC \leftarrow NBR$. Akümülatöre 500 yüklenir. Bu sayı 251 adresinden çekilir.

Dolaylı mod. Hoca "dikkat, bu baya ilginç" dedi. Doğrudan moddaki gibi 500 adresine gidilip 800 bulunur, ama 800 de bir bellek adresidir. 800 adresinden 300 okunur ve akümülatöre yüklenir: $AC \leftarrow M[M[ADRS]]$.

PC'ye göreli mod. Operandın adresi, adres alanı ile PC'nin o anki değeri toplanarak bulunur: $AC \leftarrow M[ADRS + PC]$. Peki PC ne gösterir? Komut yürütülüyorsa komut getirme adımı tamamlanmıştır. PC artık sonraki adresi, yani 252'yi gösterir. Etkin adres:

$$EA = 500 + 252 = 752$$

İndisli mod. Bir indis register'ı gerekir. Adres alanı ile bu register toplanır: $AC \leftarrow M[ADRS + \text{indis}]$. Örnekte register'ın başlangıç değeri 400'dür:

$$EA = 500 + 400 = 900$$

900 adresinden 200 okunur ve akümülatöre yüklenir.

> **Ders sonrası not:** Hoca burada 8086'dan örnek ararken CX ya da DX register'ını andı ve emin olmadığını belli etti. 8086'da indisli adreslemede büyük olasılıkla SI ve DI register'ları kullanılır. Assembly dersi notlarınızdan kontrol edin.

Register modu. Akümülatöre R_1'den yükleme yapılır: $AC \leftarrow R_1$. Bellek adresi yoktur. Yüklenen değer 400'dür.

Register dolaylı mod. Register'ın içeriği bir bellek adresini gösterir: $AC \leftarrow M[R_1]$. Bellek adresi 400'dür. Oradan okunan 700 akümülatöre yüklenir.

12.3 Özet Tablo

Mod	Register transfer	Etkin adres	AC
Doğrudan	$AC \leftarrow M[ADRS]$	500	800
Anlık	$AC \leftarrow NBR$	251	500
Dolaylı	$AC \leftarrow M[M[ADRS]]$	800	300
PC'ye göreli	$AC \leftarrow M[ADRS + PC]$	752	600 (kısa nottan)
İndisli	$AC \leftarrow M[ADRS + \text{indis}]$	900	200
Register	$AC \leftarrow R_1$	-	400
Register dolaylı	$AC \leftarrow M[R_1]$	400	700

> **Ders sonrası not:** PC'ye göreli modda 752 adresinden okunan değer altyazıda geçmiyor. Kısa not kitaptaki tabloya dayanarak 600 veriyor, yani değer büyük olasılıkla 600'dür. Tablonuzdan kontrol edin. Tablonun öteki satırları hocanın okuduğu değerlerdir ve etkin adres toplamları Python ile doğrulandı.

Soru: Bütün güncel işlemciler bu adresleme modlarını destekliyor mu? **Cevap:** Fazlası var, eksiği yok. Ama hepsi aynı işlemcinin içinde olmak zorunda değildir. Bunlar alternatif çözümlerdir ve yaklaşık yüzde 90 bunları kapsar.

13. RISC ve CISC Karşılaştırması

Bölüm 6'da sözü edilen iki mimarinin farkları slaytta şöyle özetlenir:

Özellik	RISC	CISC
Adresleme modu sayısı	Sınırlı	Oldukça fazla
Komut uzunluğu	Sabit, hepsi aynı	Değişken (1 bayt, 2 bayt ya da daha uzun)
Bellek erişimi	Yalnız load ve store ile	İşlemlerde doğrudan bellek operandı
İşlemler	Temel işlemler	Hem temel hem karmaşık işlemler
Kontrol	Kablolu (hardwired), hızlı	Mikroprogramlı, esnek, daha yavaş

RISC'te bir ADD işleminin operandını doğrudan bellekten alamazsınız. Önce load ile iç register'a taşır, sonra işleme katarsınız. CISC'te böyle bir zorunluluk yoktur. 8086'da bellekteki bir sayı ile doğrudan işlem yapabilirsiniz.

Kontrol tarafında mikroprogramlı yapı daha esnektir, üzerinde oynanabilir. Kablolu yapının avantajı ise daha hızlı çalışmasıdır. Hoca bunu "avantajları var, dezavantajları var" diye bağladı.

> **Ders sonrası not:** Hoca karşılaştırmanın ortasında "CISC tarafına baktığımız zaman adresleme modlarının sayısı sınırlı ve komut uzunlukları sabit" dedi. Bu RISC'in özelliğidir. Hemen ardından söylediği "çok karmaşık komutlar yok, temel işlemler var" gerekçesi de bunu gösterir.

14. Veri Transfer Komutları ve Giriş-Çıkış Adresleme

Şimdi bu mimarilerde bulunabilecek komutlara geçiyoruz. Assembly bilenler bunların çoğunu tanır, ama bilmedikleri komutlar da olabilir.

Veri transferi bellek ile register arasında ya da işlemci register'ları ile giriş-çıkış (G/Ç) birimleri arasında olabilir. Temel komutlar şunlardır:

- **LD (load):** İç register'a yükleme.
- **ST (store):** Register içeriğini belleğe aktarma.
- **MOVE:** 8086'da bulunan transfer komutu.
- **XCHG (exchange):** İki register'ın içeriğini değiştirme.
- **PUSH, POP:** Yığın üzerindeki işlemler (Bölüm 11).
- **IN, OUT:** G/Ç portlarına erişim.

Hoca exchange komutunun tek adımda gerçekleşmediğini, arka planda bir mikroprogramın çalıştığını anlattı. Önce R_1'in içeriği geçici bir alana kaydedilir, sonra R_2 R_1'e kaydedilir, sonra geçici alandaki değer R_2'ye konur. Python benzetimi bu üç adımın takası doğru yaptığını gösterdi.

14.1 Giriş-Çıkış Komutları

G/Ç komutları, işlemci register'ları ile G/Ç birimlerinin portları arasında veri taşır. Klavye ve fare giriş birimlerine, yazıcı çıkış birimine örnektir. Farklı mimarilerde bu işlemler iki farklı şekilde kurulur.

14.2 Bağımsız G/Ç (Independent ya da Isolated I/O)

Belleğe ve G/Ç portlarına atanan adres alanları birbirinden bağımsızdır. Intel 8086 böyle çalışır.

Bellek alanı. İşlemcinin adres yoluyla adresleyebildiği aralıktır. 8086'da 20 adres ucu olduğu için bellek alanı 2^{20} bayt, yani 1 MB'tır. Bu alana 8086'da MOV komutuyla erişilir.

G/Ç alanı. Ayrı bir alandır ve IN, OUT komutlarıyla erişilir. Adres uçları iki şekilde kullanılabilir:

- `IN AL, #port`: Port numarası doğrudan verilir ve o porttan AL'ye okunur. Port numarası 8 bitle sınırlıdır. Dolayısıyla G/Ç alanı $2^8 = 256$ bayttır.
- `IN AL, DX`: Port DX üzerinden adreslenir. DX 16 bit olduğu için G/Ç alanı 2^{16} bayt, yani 64 KB olur.

Hoca ikinci yazılış için "DX 16 bit, AL 8 bit, hocam ne yaptınız demeyin" diye uyardı. DX yalnız adresi gösterir. Yapılan veri transferi yine 8 bittir. DX ile G/Ç alanı büyür, okunan veri büyümez.

G/Ç alanı, bellek alanı gibi RAM entegreleri dizilerek kurulmuş bir yapı değildir. Anakartta "şu entegre bu alanı tutuyor" diye gösteremezsiniz. Yalnız belirli adresleri çözen adres çözme devreleri vardır. Ayrıntıları mikroişlemciler dersinde görülecek.

14.3 Belleğe Eşlenmiş G/Ç (Memory-Mapped I/O)

Ayrı bir G/Ç alanı yoktur. Tek bir adres alanı vardır. Bu alanın belli bir bölgesi G/Ç için, belli bir bölgesi bellek için kullanılır.

15. Veri İşleme Komutları

Veri üzerinde işlem yapan komutlar aritmetik, lojik ve kaydırma olarak gruplanır.

Aritmetik: artırma (INC), eksiltme (DEC), toplama (ADD), çıkarma (SUB), çarpma (MUL), bölme (DIV), elde ile toplama (add with carry), ödünç ile çıkarma (subtract with borrow), ters çıkarma (subtract reverse, $A - B$ yerine $B - A$) ve negatifini alma (negate). Hoca çarpma ve bölmenin her mimaride olmayabileceğini özellikle belirtti.

Lojik: clear, set, complement, AND, OR, XOR, clear carry, set carry, complement carry.

Kaydırma: Dört grupta toplanır: lojik kaydırma, aritmetik kaydırma, döndürme (rotate) ve elde ile döndürme (rotate with carry).

15.1 Lojik Kaydırma

Lojik kaydırma, seri girişten gelen verinin hep 0 olması demektir. Sağa kaydırmada sayının yüksek anlamlı ucundan 0 girer. Sola kaydırmada da seri giriş 0'dır. Dışarı atılan bit ise carry'ye konur. Bu standarttır.

Soru (hocanın sorusu): Bir sayıyı bir bit sağa kaydırmak sayının değerine ne yapar? **Cevap (bir öğrenci):** İkiye böler ve aşağı yuvarlar. Son bit dışarı atıldığı için yuvarlama olur.

Sola kaydırmak ise 2 ile çarpmak demektir.

15.2 Aritmetik Kaydırma

Bu yorum işaretli sayılarda önem kazanır.

Aritmetik sağa kaydırma. Sayı ikiye bölünür, yani yalnız küçülür. İşareti değişmemelidir. Bu yüzden işaret biti alınıp kendi üzerine yeniden yüklenir.

Aritmetik sola kaydırma. Hocanın "sıkıntılı nokta" dediği yer burasıdır. Sayı işaretliyse işaret biti dışarı taşabilir ve sayı artık 8 bitlik register'da ifade edilemeyebilir. Sola kaydırmak 2 ile çarpmak demektir, sayı büyür. Bu yüzden taşma kontrolü gerekir.

Taşmayı şöyle kontrol ederiz: 8 bitlik register'da en yüksek anlamlı bit olan 7. bit işareti tutar. Sola kaydırınca 6. bit, 7. bitin yerine gelir. Bu iki bit birbiriyle aynıysa taşma olmaz, farklıysa taşma olur. "Aynıysa 0, farklıysa 1" davranışını XOR kapısı verir. Kaydırmadan **önceki** bitlerle:

$$V = R_7 \oplus R_6$$

> **Ders sonrası not:** Hoca bir yerde "taşma olur", bir yerde "oluşabilir" dedi. Python ile 8 bitlik bütün 256 değer denendi: $R_7 \neq R_6$ olan her değerde taşma kesin olur, $R_7 = R_6$ olan hiçbir değerde olmaz.
>
> **Hocanın vurgusu:** Aritmetik kaydırmada yalnız sola kaydırma sırasında taşma kontrolü gerekir. "Bunu bilmenizi istiyorum."

15.3 Döndürme

Sağa döndürmede en düşük anlamlı bitten atılan değer en yüksek anlamlı bite geri yüklenir. Sola döndürmede en yüksek anlamlı bit en düşük anlamlı bitin yerine gelir.

Elde ile döndürmede carry de zincire katılır. Sağa doğru bir işlemde en düşük anlamlı bit C'ye, C'nin değeri register'ın en yüksek anlamlı bitine yerleştirilir. Hoca bu yapıyı gözünüzün önüne getirmenizi tavsiye etti.

16. Program Kontrol Komutları

Program kontrol komutları branch, jump, call, return, compare ve test komutlarıdır. Bunlarda koşullu ya da koşulsuz dallanma vardır. Jump ile PC bir adresten başka bir adrese taşınır. Call ile bir alt program çağrılır. Alt program bitince geri dönmek için son satırına bir return konur.

Akışı kontrol etmek için kullanılan iki komut sonucu hiçbir register'a yazmaz, yalnız durum bitlerini günceller:

- **Compare (`CMP R1, R2`):** İki register'ı $R_1 - R_2$ çıkarmasıyla karşılaştırır.
- **Test (`TEST R1, R2`):** R_1 ve R_2 arasında AND yapar. Örneğin AND sonucunda bütün bitler 0 çıkıyorsa Z biti 1 olur.

17. Koşullu Dallanma Komutları ve Durum Bitleri

Hoca bu konu için "en önemli konu en sona kaldı" dedi.

Temel koşullu dallanma komutları tek bir duruma bakar:

Komut	Dallanma koşulu
Branch if zero	$Z = 1$
Branch if not zero	$Z = 0$
Branch if carry	$C = 1$
Branch if minus	$N = 1$ (sayı negatif)
Branch if overflow	$V = 1$

İki sayıyı karşılaştırma tabloları ise işaretsiz ve işaretli sayılar için ayrıdır.

- **İşaretsiz sayılar:** 8 bitlik register 0-255 arasındaki bütün değerleri tutar. İşaret biti yoktur. Bu yüzden N ve V gerekmez. Yalnız C ve Z kullanılır.
- **İşaretli sayılar:** En yüksek anlamlı bit işaret bitidir. Değerler -128 ile $+127$ arasındadır. Sıfır, işareti pozitifmiş gibi göründüğü için pozitif taraf 128'e değil 127'ye kadar gider. Burada N, V ve Z önemlidir.

17.1 İşaretsiz Sayılar ve Ödünç Biti

> **Hocanın vurgusu:** Tablonun altındaki dipnota dikkat edin. Tabloda C diye görülen değer aslında ödünç (borrow) bitidir. Ödünç biti, carry bitinin tümleyenidir. C'yi buna göre değerlendirmelisiniz.

$A - B$ yapıldıktan sonra koşullar şöyledir:

- $A > B$ (branch if above): $C + Z = 0$. Ödünç alınmamış ve sonuç sıfır değildir.
- $A \geq B$ (branch if higher or equal): $C = 0$.
- $A < B$ (branch if lower): $C = 1$.

17.2 İşaretli Sayılar: $A > B$ Koşulu Neden Böyle?

Tablo işaretli sayılarda $A > B$ için şu koşulu verir:

$$(N \oplus V) + Z = 0$$

Hoca bunun nedenini anlattı. Önce `CMP A, B`, ardından branch if greater (BG) yazılır. Yani N, V ve Z bitleri önceki komutta $A - B$ yapılarak güncellenmiştir. $A - B$ pozitifse A büyüktür. Ama işaret bitine güvenmeden önce taşma var mı diye bakmak gerekir. Dört durum vardır:

N	V	$N \oplus V$	Yorum	Sonuç
0	0	0	Sonuç pozitif, taşma yok	$A > B$ ($Z = 0$ ise)

N	V	$N \oplus V$	Yorum	Sonuç
0	1	1	Pozitif görünüyor, ama gerçek işaret biti dışarı taştı. Gerçek işaret 1'dir	$A < B$
1	0	1	Sonuç negatif, taşma yok	$A < B$
1	1	0	Negatif görünüyor, ama işaret biti doğru değil. Sayı aslında pozitiftir	$A > B$

$N \oplus V = 0$ ve $Z = 0$ ise $A > B$ denebilir. Z bu formüle $A = B$ durumunu ayırmak için girer.

> **Ders sonrası not:** 8 bitlik bütün 65.536 (A, B) çifti Python ile denendi. Yukarıdaki dört durum ve formüller hepsinde doğru sonuç verdi. $N = 0, V = 0$ satırının kesin karşılığı $A \geq B$'dir, $A > B$ için ayrıca $Z = 0$ gerekir.
>
> **Hocanın vurgusu:** Amaç yalnız tabloyu kullanmak değil, tablonun bize ne anlattığını ve nasıl oluştuğunu anlamaktır.
>
> **Ek bilgi (derste anlatılmadı):** Bu tabloların hoca tarafından okunmayan satırları, okunan satırların tümleyeninden elde edilir. İşaretsizde $A \leq B$ için $C + Z = 1$, işaretlide $A \geq B$ için $N \oplus V = 0$, $A < B$ için $N \oplus V = 1$, $A \leq B$ için $(N \oplus V) + Z = 1$ yazılır. Her ikisinde de $A = B$ için $Z = 1$, $A \neq B$ için $Z = 0$ yazılır. Bu formüller 65.536 çiftin tamamında Python ile doğrulandı.

18. Çözümlü Örnek 1: İşaretsiz Karşılaştırma

Kitabın bölüm sonu problemlerinden biridir: Bir program, iki işaretsiz sayıyı $A - B$ çıkarmasıyla karşılaştırıp durum bitlerini güncelliyor. Sayılar 8 bit olarak veriliyor: $A = 93$, $B = 92$.

(a) Farkı hesaplayın ve ikili sonucu yorumlayın. İkili olarak $A - B$, A ile B'nin ikiye tümleyeninin toplamıdır.

İkiye tümleyen için kestirme yol: Sağdan başlanır, ilk 1 görülene kadar (o 1 dâhil) bitler aynen yazılır, sonrakilerin tümleyeni alınır. $B = 01011100$ için sağdaki `100` aynen kalır, kalan `01011` bitleri `10100` olur: 10100100 (ara adım eklendi).

```
A     =   0 1 0 1 1 1 0 1    (93)
-B    = + 1 0 1 0 0 1 0 0    (92'nin ikiye tümleyeni)
--------------------------
elde 1    0 0 0 0 0 0 0 1    (1)
```

Sonuç 1, carry biti 1 çıktı. Carry'nin 1 olması, ödünç bitinin 0 olması demektir. Yani $A - B$ yapılırken borç almak gerekmedi. Sağlama: $93 - 92 = 1$.

(b) Durum bitlerini bulun. Ödünç biti 0'dır. Z biti, sonuçta bütün bitler 0 olsaydı 1 olurdu. Burada öyle değildir, bu yüzden $Z = 0$.

(c) Koşulu doğru olan dallanma komutlarını listeleyin (kitaptaki Tablo 11.9). Tablodaki bütün C'ler ödünç biti olarak okunur:

Dallanma	Koşul	Değerlendirme	Sonuç
$A > B$ (above)	$C + Z = 0$	$0 + 0 = 0$	Doğru
$A \geq B$ (higher or equal)	$C = 0$	$C = 0$	Doğru
$A < B$ (lower)	$C = 1$	$C = 0$	Yanlış
$A \leq B$ (lower or equal)	$C + Z = 1$	$0 + 0 = 0$	Yanlış
$A = B$ (equal)	$Z = 1$	$Z = 0$	Yanlış

Son iki satırın sonucunu hoca söyledi, koşul sütunu ara adım olarak eklendi.

19. Çözümlü Örnek 2: Aritmetik Kaydırma ve İşaretli Karşılaştırma

Bu örnek eski bir sınav sorusudur ve iki şıktan oluşur.

19.1 (a) Aritmetik Kaydırma

Soru: $+62$ ve -75 sayılarını 8 bitlik işaretli ikili sayıya çevirin. Aritmetik sola ve sağa kaydırma yapıp taşma bitini gösterin.

$+62$ **için.** $62 = 00111110$. Sola kaydırmadan önce işaret biti ile yanındaki bite bakılır: $R_7 = 0$, $R_6 = 0$. İkisi aynı olduğu için kaydırma sonunda işaret değişmez ve $V = 0$ çıkar. Sonuç $01111100 = +124$. Sağa kaydırmada sonuç $00011111 = +31$ olur (ara sonuçlar eklendi).

Hoca sağa kaydırmada taşma kontrolüne gerek bile olmadığını hatırlattı: İkiye bölünen sayı küçülür, büyümez. Taşma riski yalnız sola kaydırmada vardır.

-75 **için.** Önce -75 doğru yazılmalıdır.

> **Hocanın vurgusu:** Burada çok hata yapılıyor. Önce $+75$ ikili olarak yazılır, sonra ikiye tümleyeni alınarak -75 bulunur. Çok görülen hata, yalnız en yüksek anlamlı biti 1 yapıp sayıyı -75 yaptığını sanmaktır. Böyle olmuyor.

$$+75 = 01001011 \quad \Rightarrow \quad \text{tümleyen } 10110100 \quad \Rightarrow \quad +1 \quad \Rightarrow \quad -75 = 10110101$$

Değeri geri bulmak için de aynı mantık kullanılır: İşaret biti 1 ise ikiye tümleyen alınır, sayı değeri bulunur.

Sola kaydırmadan önce $R_7 = 1$, $R_6 = 0$. İkisi farklı olduğu için $V = 1$. Kaydırınca 0 en yüksek anlamlı bite gelir ve sonuç 01101010 olur. Sayı pozitif görünür ($+106$). Yani bu sonuç artık aritmetik olarak anlamlı değildir, taşma vardır.

Sağa kaydırmada işaret biti kendi üzerine yeniden yüklenir: $11011010 = -38$. Bu, $-75/2 = -37{,}5$ değerinin aşağı yuvarlanmış hâlidir (ara sonuç eklendi).

Sayı	İkili	Sola kaydırma	V	Sağa kaydırma
$+62$	00111110	01111100 ($+124$)	0	00011111 ($+31$)
-75	10110101	01101010 ($+106$, anlamsız)	1	11011010 (-38)

19.2 (b) İşaretli Karşılaştırma

Soru: Bir program iki işaretli (ikiye tümleyen) sayıyı $A - B$ çıkarmasıyla karşılaştırıyor. Sonucu değerlendirin ve yorumlayın. Karşılaştırmada sonuç hiçbir yere yazılmaz, yalnız durum bitleri güncellenir.

Taşma, işaret bitine giren elde ile işaret bitinden çıkan eldenin XOR'udur.

Birinci çift: $A = -34$**,** $B = +94$**.** A'nın en yüksek anlamlı biti 1 olduğu için negatiftir. B'nin ikiye tümleyeni alınıp A ile toplanır:

```
elde     1 1 1 1 1 1 1 0 0
A      =   1 1 0 1 1 1 1 0    (-34)
-B     = + 1 0 1 0 0 0 1 0    (-94)
------------------------------
           1 0 0 0 0 0 0 0    (-128)
```

İşaret bitine giren elde 1, çıkan elde 1'dir. $V = 1 \oplus 1 = 0$. Taşma yoktur. Gerçekten $-34 - 94 = -128$ ve -128 8 bitle gösterilebilen sınırın içindedir. Sonuç negatif olduğu için $N = 1$, sıfır olmadığı için $Z = 0$.

$A > B$ koşulu sınanır:

$$(N \oplus V) + Z = (1 \oplus 0) + 0 = 1 \neq 0$$

Koşul sağlanmaz, sonuç **yanlış**tır. Doğrusu da budur: -34, 94'ten büyük değildir.

İkinci çift: $A = +82$**,** $B = -46$**.** B negatif olduğu için $A - B$ işleminde $-(-46) = +46$ toplanır. Gerçek sonuç $+128$'dir ve bu 8 bitlik register'da ifade edilemez. Hoca bu sayıların sınırı göstermek için özellikle seçildiğini söyledi.

```
elde     0 1 1 1 1 1 1 0 0
A      =   0 1 0 1 0 0 1 0    (+82)
-B     = + 0 0 1 0 1 1 1 0    (+46)
------------------------------
           1 0 0 0 0 0 0 0    (+128 sığmaz)
```

İşaret bitine giren elde 1, çıkan elde 0'dır. $V = 1 \oplus 0 = 1$, yani taşma oluşur. $Z = 0$ ve $N = 1$ çıkar. Burada ilginç bir durum vardır: İki pozitif sayı toplanıyor, ama sonuç negatif görünüyor. Çünkü işaret biti 8 bitlik register'da tutulamıyor.

> **Ders sonrası not:** Hoca "$A > B$ sonucu false çıkmış" sonucunu yalnız birinci çift için söyledi. İkinci çift için formül $(1 \oplus 1) + 0 = 0$ verir, yani $A > B$ **doğrudur** ve gerçekten $82 > -46$'dır. Taşma olsa bile karşılaştırma doğru sonuç verir. İki çiftin bütün bitleri Python ile doğrulandı.

20. Ders Sonu Sorusu: Onluk Tabanda Tümleyen

Soru: İkiye tümleme mantığı onluk tabanda da aynı değil mi? Örneğin $4 - 2$ için 2'nin onluk tabandaki tümleyeni 8'dir. $4 + 8 = 12$ olur, taşan kısım atılınca sonuç 2 çıkar. **Cevap:** Öyle düşünebilirsiniz. Bunu farklı şekillerde mantığınıza yerleştirmeniz gerekir.

> **Ders sonrası not:** Altyazıda öğrencinin "7 bitte düşününce 256'ya tamamladım" dediği okunuyor. 256, 2^8'dir, yani doğrusu 8 bittir. Benzetmenin kendisi doğrudur: $(4 + 8) \bmod 10 = 2$.

Dersin Toparlaması

Dersin ilk bölümünde register'lar arasında veri taşımanın üç yolunu karşılaştırdık. Ayrılmış çoklayıcı esnektir ama pahalıdır. Tek veri yolu ucuzdur ama aynı anda tek kaynak taşır. Üç durumlu veri yolunda çoklayıcı aradan çıkar, kaynağı etkinleştirme uçları seçer ve bu uçlardan yalnız birinin 1 olmasını bir decoder garanti eder. Bu son yapı, ileride tasarlayacağımız işlemcide kullanılacaktır.

İkinci bölümde işlemcinin programcıya görünen yüzünü, yani komut kümesi mimarisini gördük: RISC ve CISC, komut biçimi, IF-ID-OF-EX-WB çevrimi, adres sayısına göre komutlar, yığın, adresleme modları ve komut türleri. Son olarak `CMP` ile güncellenen durum bitlerinden koşullu dallanma kararının nasıl çıktığını çözdük. Komut çevrimi adımları pipeline konusunda, veri yolu ve durum bitleri de CPU tasarımında yeniden karşımıza çıkacak. Kitabın bu bölümünün sonundaki benzer sorular konuyu pekiştirmek için çözülebilir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.